

NODE.JS BASICS

Beginner Tutorial with REST APIs, Status Codes and MongoDB

Goal: Learn Node.js from basic commands to building REST APIs with Express and MongoDB using simple examples.

You will learn	Practical output
Node.js basics	Run JavaScript outside the browser
npm and modules	Install and use packages
Express.js	Create server and routes
REST APIs	GET, POST, PUT, DELETE endpoints
HTTP status codes	Send correct success and error responses
MongoDB + Mongoose	Store, fetch, update and delete data

Table of Contents

1. What is Node.js?
2. Setup and first program
3. Modules and npm
4. Express.js basics
5. REST API concepts
6. HTTP status codes
7. MongoDB and Mongoose
8. Complete Product CRUD API
9. Testing APIs
10. Practice tasks and viva questions

1. What is Node.js?

Node.js is a runtime environment that allows JavaScript to run outside the browser. It is commonly used to create backend servers, APIs, command-line tools and real-time applications.

Term	Simple meaning
Runtime	A place where code runs
Backend	Server-side part of an application
API	A way for frontend and backend to communicate
npm	Package manager for Node.js

Example: A React/Ionic app can call a Node.js API to get products from MongoDB.

2. Setup and First Program

Install Node.js from the official website, then check the version in terminal.

```
node -v
npm -v
```

Create a file named app.js and run it using node.

```
console.log('Hello Node.js');

// Run
node app.js
```

3. Modules and npm

A module is a reusable file or package. Node.js has built-in modules and also supports third-party packages from npm.

```
// Built-in module example
const fs = require('fs');

fs.writeFileSync('demo.txt', 'Hello file system');
console.log('File created');
```

```
npm init -y
npm install express mongoose dotenv
```

Package	Use
express	Create server and APIs
mongoose	Connect Node.js with MongoDB

dotenv	Read environment variables from .env file
nodemon	Auto-restart server during development

4. Express.js Basics

Express.js is a popular Node.js framework used to create web servers and APIs easily.

```
const express = require('express');
const app = express();

app.get('/', (req, res) => {
  res.send('Server is running');
});

app.listen(5000, () => {
  console.log('Server started on port 5000');
});
```

Part	Meaning
app	Express application
app.get()	Route for GET request
req	Request sent by client
res	Response sent by server
listen()	Starts server on a port

5. REST API Concepts

REST API is a standard way to create backend endpoints using HTTP methods. It usually works with JSON data.

HTTP Method	Use	Example endpoint
GET	Read data	GET /api/products
GET	Read one item	GET /api/products/101
POST	Create new data	POST /api/products
PUT/PATCH	Update data	PUT /api/products/101
DELETE	Delete data	DELETE /api/products/101

REST rule: Use nouns in URLs like /products, not actions like /getProducts.

Simple REST API without Database

```
const express = require('express');
const app = express();
app.use(express.json());
```

```

let products = [
  { id: 1, name: 'Keyboard', price: 800 },
  { id: 2, name: 'Mouse', price: 400 }
];

app.get('/api/products', (req, res) => {
  res.status(200).json({ success: true, products });
});

app.post('/api/products', (req, res) => {
  const product = { id: Date.now(), ...req.body };
  products.push(product);
  res.status(201).json({ success: true, product });
});

app.listen(5000, () => console.log('API running'));

```

6. HTTP Status Codes

Status codes tell the client whether the request was successful or failed. Correct status codes make APIs professional and easy to debug.

Code	Meaning	When to use
200 OK	Success	Data fetched successfully
201 Created	New resource created	Product/user created
204 No Content	Success but no response body	Deleted successfully without returning data
400 Bad Request	Invalid input	Required field missing
401 Unauthorized	Login required	No token or invalid token
403 Forbidden	Access denied	User logged in but not allowed
404 Not Found	Resource not found	Product ID does not exist
409 Conflict	Duplicate/conflict	Email already registered
500 Internal Server Error	Server problem	Database/server crash

```

// Example: validation error
if (!req.body.name) {
  return res.status(400).json({
    success: false,
    message: 'Product name is required'
  });
}

// Example: not found
if (!product) {
  return res.status(404).json({
    success: false,
    message: 'Product not found'
  });
}

```

```
}
```

7. MongoDB and Mongoose

MongoDB is a NoSQL database that stores data in document format. Mongoose is a Node.js library used to create schemas and perform database operations easily.

MongoDB word	Simple meaning
Database	Collection of data
Collection	Like a table
Document	Like one record/row
Field	A property inside a document
_id	Unique ID automatically created by MongoDB
Schema	Structure/rules of a document

MongoDB Connection

```
// .env file
PORT=5000
MONGO_URI=mongodb://127.0.0.1:27017/node_tutorial

// db.js
const mongoose = require('mongoose');

async function connectDB() {
  try {
    await mongoose.connect(process.env.MONGO_URI);
    console.log('MongoDB connected');
  } catch (error) {
    console.error('MongoDB connection failed', error.message);
    process.exit(1);
  }
}

module.exports = connectDB;
```

Mongoose Model

```
// models/Product.js
const mongoose = require('mongoose');

const productSchema = new mongoose.Schema({
  name: { type: String, required: true, trim: true },
  price: { type: Number, required: true, min: 0 },
  category: { type: String, default: 'General' },
  inStock: { type: Boolean, default: true }
}, { timestamps: true });
```

```
module.exports = mongoose.model('Product', productSchema);
```

8. Complete Product CRUD API

CRUD means Create, Read, Update and Delete. This is the most common backend API pattern.

```
// server.js
require('dotenv').config();
const express = require('express');
const connectDB = require('./db');
const productRoutes = require('./routes/productRoutes');

const app = express();
app.use(express.json());

connectDB();

app.get('/', (req, res) => {
  res.send('Node REST API with MongoDB');
});

app.use('/api/products', productRoutes);

const PORT = process.env.PORT || 5000;
app.listen(PORT, () => console.log(`Server running on ${PORT}`));
```

Product Routes

```
// routes/productRoutes.js
const express = require('express');
const Product = require('../models/Product');
const router = express.Router();

// GET all products
router.get('/', async (req, res) => {
  try {
    const products = await Product.find().sort({ createdAt: -1 });
    res.status(200).json({ success: true, count: products.length, products });
  } catch (error) {
    res.status(500).json({ success: false, message: 'Unable to fetch products' });
  }
});

// GET single product
router.get('/:id', async (req, res) => {
  try {
    const product = await Product.findById(req.params.id);
    if (!product) {
      return res.status(404).json({ success: false, message: 'Product not found' });
    }
  }
});
```

```

    }
    res.status(200).json({ success: true, product });
  } catch (error) {
    res.status(400).json({ success: false, message: 'Invalid product ID' });
  }
});

// CREATE product
router.post('/', async (req, res) => {
  try {
    const { name, price, category } = req.body;
    if (!name || price === undefined) {
      return res.status(400).json({ success: false, message: 'Name and price are required' });
    }
    const product = await Product.create({ name, price, category });
    res.status(201).json({ success: true, product });
  } catch (error) {
    res.status(500).json({ success: false, message: 'Unable to create product' });
  }
});

// UPDATE product
router.put('/:id', async (req, res) => {
  try {
    const product = await Product.findByIdAndUpdate(req.params.id, req.body, {
      new: true,
      runValidators: true
    });
    if (!product) {
      return res.status(404).json({ success: false, message: 'Product not found' });
    }
    res.status(200).json({ success: true, product });
  } catch (error) {
    res.status(400).json({ success: false, message: 'Invalid update request' });
  }
});

// DELETE product
router.delete('/:id', async (req, res) => {
  try {
    const product = await Product.findByIdAndDelete(req.params.id);
    if (!product) {
      return res.status(404).json({ success: false, message: 'Product not found' });
    }
    res.status(200).json({ success: true, message: 'Product deleted successfully' });
  } catch (error) {
    res.status(400).json({ success: false, message: 'Invalid product ID' });
  }
});

module.exports = router;

```

API Testing Examples

Task	Method	URL	Body
Get all products	GET	http://localhost:5000/api/products	-
Get one product	GET	http://localhost:5000/api/products/:id	-
Create product	POST	http://localhost:5000/api/products	{ "name": "Laptop", "price": 55000 }
Update product	PUT	http://localhost:5000/api/products/:id	{ "price": 52000 }
Delete product	DELETE	http://localhost:5000/api/products/:id	-

9. Folder Structure

```
node-rest-api/  
  server.js  
  db.js  
  .env  
  package.json  
  models/  
    Product.js  
  routes/  
    productRoutes.js
```

Important: Never upload .env to GitHub. Add it in .gitignore because it may contain passwords or secret keys.

10. Practice Tasks

- Create a /api/students GET API that returns a list of students.
- Create a POST /api/students API to add a new student.
- Add validation: name and rollNo are required.
- Create a Student model using Mongoose.
- Add 404 response when student ID is not found.
- Test all APIs using Postman or Thunder Client.

Simple Viva Questions

Question	Simple answer
What is Node.js?	It runs JavaScript outside the browser.
What is npm?	It installs and manages Node packages.
What is Express?	A framework to create APIs and servers.
What is REST API?	A standard way for frontend and backend to communicate using HTTP methods.

What is MongoDB?	A NoSQL database that stores data as documents.
What is Mongoose?	A library to connect Node.js with MongoDB using schemas.
What is status code 404?	Resource not found.
What is status code 201?	New data created successfully.